A Formal Approach to Modeling and Analyzing Human Taskload in Simulated Air Traffic Scenarios

Adam Houser, Lanssie Mingyue Ma, Karen Feigh *Senior Member*, *IEEE* and, Matthew L. Bolton, *Member*, *IEEE*

***Abstract*—In complex systems, like the modern air traffic system, human operator taskload can have a profound influence on how well the system performs. Because of the system's complexity, however, it can be difficult to determine all of the situations where taskload issues can arise. Simulation and formal verification have been used separately to explore human taskload in complex systems. However, both have problems that limit their usefulness. In this paper, we describe a formal modeling architecture designed to enable the discovery of interesting human operator taskload conditions though the synergistic use of formal verification and simulation. This architecture formally represents original simulation constructs using computationally efficient abstractions that ensure that temporal and ordinal relationships between simulation events (actions) are represented realistically. Taskload for each agent is represented based on a priority queue model where only a limited number of actions can be performed or remembered by a human at any given time. We provide an overview of this architecture, discuss its essential features, and describe the mathematical foundations needed for its instantiation. We present insights into its capabilities for finding interesting taskload conditions by formulating several checkable specification properties. The implications of this architecture are discussed in terms of its broader supported analysis method and directions for future work are explored.**

***Index Terms*—Formal methods, taskload, workload, simulation, human-automation interaction.**

I. Introduction

HUMAN operator taskload, a measure of the number of tasks a human operator is expected to perform at a given time, is critical to the safe and efficient operations of complex systems such as the air traffic system. This is because taskload is a good indicator of human operator workload [**?**], where excessive taskload/workload leads to human error and reduced performance. However, determining when taskload can become excessive and what the performance implications of that taskload are can be very challenging because of the many different people, machines, and environmental conditions that can interact during their operation. Running experiments or tests with real world systems and human subjects can be too time consuming and expensive to explore the many operating conditions that can occur. To address this, researchers have been building simulation environments such as Work Models that Compute (WMC) [**?**] that allow human operator taskload to be analyzed in a variety of air traffic simulations. While more flexible than human subject experiments and real world tests, these simulation types are not exhaustive and can thus still miss potentially dangerous or performance-critical operating conditions that did not happen to occur in one of the explored scenarios. Formal verification represents an analysis approach that specifically addresses this limitation of simulation.

Adam Houser is with the Department of Systems and Industrial Engineering, University at Buffalo, State University of New York, Amherst, NY

Lanssie Mingyue Ma is with the Daniel Guggenheim School of Aerospace Engineering in Computational Science and Engineering, Georgia Institute of Technology, Atlanta, GA

Karen Feigh is with the Daniel Guggenheim School of Aerospace Engineering, Georgia Institute of Technology, Atlanta, GA

Matthew L. Bolton is with the Department of Systems and Industrial Engineering, University at Buffalo, State University of New York, Amherst, NY 14260 USA e-mail: mbolton@buffalo.edu

A. Formal Verification

Formal verification is an analysis technique that falls within the discipline of formal methods. Formal methods are well-defined mathematical languages and techniques for the specification, modeling, and verification of systems [**?**]. Specification properties mathematically describe desirable system conditions. Systems are modeled using mathematically-based languages, and verification then mathematically proves whether or not the model satisfies the specification. Model checking is an automated approach to formal verification [**?**], whereby a formal model describes a system as a state transition model: a set of variables and transitions between variable states. Desirable specification properties are usually represented in a temporal logic [**?**]. Verification is performed automatically by exhaustively searching a system's statespace to determine if these properties hold. If they do, the model checker returns a confirmation. Otherwise, a counterexample is produced, which shows how the specification violation occurred as a trace through the statespace of the model.

Formal verification has been successfully used to evaluate human-automation interaction [**?**]. However, little work has been done to investigate human operator workload or taskload. While Mercer and Goodrich, et al. ([**? ?**]) have investigated ways of formally modeling workload, they have not used these methods in formal verification analyses.

While powerful, formal verification techniques such as model checking suffer from combinatorial explosion, where the statespace grows exponentially as additional components are added to the model [**?**]. This can quickly lead to a model that is too big to be verified. Model checking is also limited by the expressive power of its notations, where models cannot contain non-linear arithmetic or other typical programming constructs (such as loops or type casting). As such, formal verification scales badly when compared to simulation and is limited in what system behavior it can consider.

B. Formal Verification and Simulation

Some degree of success has been found in using formal verification synergistically with simulation to exploit the

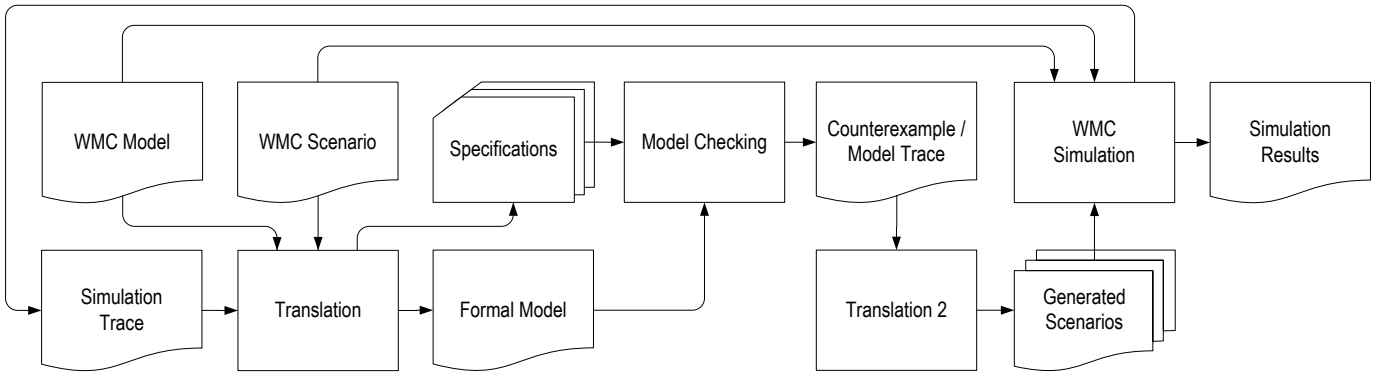

Fig. 1. Method for the synergistic use of WMC simulation and model checking.

exhaustive capabilities of model checking with the scalability of simulation. Specifically, formal verification is used selectively to evaluate bounded elements of a simulated system [**? ? ?**]. Of particular interest to this project is work that has used simulation traces as a means of creating formal models of a scope small enough to avoid scalability problems (for example, see [**? ? ?**]). However, these analyses are limited in that they only check properties about the actual trace and thus do not account for any system behavior beyond what is already in the trace. There is therefore a need to use formal verification and simulation together more effectively, employing formal verification to explore the space around simulation traces rather than just the traces themselves.

C. Our Method

We are developing an analysis approach that will allow the WMC simulation to be used synergistically with formal verification. Specifically, we want to give analysts the ability to use model checking's exhaustive search capabilities to explore the region around a simulated air traffic scenario and find excessive human taskload conditions worthy of deeper, high-fidelity simulation analysis. To accomplish this, our method uses the Symbolic Analysis Laboratory (SAL) [**?**] to model check the space around WMC simulation traces.

This method (see Fig. 1) works as follows:

1) A WMC work model (which describes the agents in a simulation, along with the actions they perform and the resources they modify) and a scenario (which describes the initial conditions that represent a specific air traffic situation and future events that can occur) is run through a WMC simulation. The simulation produces a trace showing exactly how that scenario evolved.
2) The work model, scenario, and simulation trace are then automatically translated into a formal model representing the simulation over a constrained period of time. In this model, the timing of actions (when they occur and for how long they occur) can include analyst-defined variance, nominally on the order of one to three seconds, to allow the model checker to explore the performance space around the modeled scenario. The translator also generates a set of specification properties designed to find interesting taskload conditions in the model.
3) A model checker is then used to explore the formal model to generate traces illustrating violations of specifications.
4) The traces are then translated back into WMC scenarios for deeper analyses in its simulation environment.

D. Objectives

To be able to formally model WMC concepts with our method (Fig. 1), we needed a formal modeling architecture. The architecture needed to support all of the following: (a) *Modeling real-valued time:* Because our method allows analysts to evaluate how variance in timing affects taskload, we needed the capability to formally model real-valued time; (b) *Modeling taskload:* WMC can support a priority-queue-based approach to modeling human taskload and control how humans switch between tasks and actions [**?**]. Thus, our architecture needed to be able to replicate the taskload and task switching behavior of WMC; (c) *Computational efficiency:* Because of the scalability limitations of model checking, the architecture must represent WMC concepts in a computationally efficient manner. In this paper, we describe a formal architecture that satisfies these requirements. We first discuss the relevant WMC concepts the architecture needed to encapsulate. We then discuss our architecture and how it has been realized. We also describe the specification properties that we can use with the model to generate traces. Finally, we discuss how we plan to use our architecture in future evaluations of air traffic scenarios.

II. WMC

Work Models that Compute (WMC) is a simulation framework that dynamically models complex, multi-agent concepts of operations and work domains [**?**]. WMC attempts to model the collective work of a set of agents [**?**]. It consists of two parts: a work model that describes the work of a given domain, and an engine that simulates the work model [**?**]. Each work model is comprised of three primary elements: agents, actions, and resources. Resources are defined as a collection of specific elements of the work environment which can be sensed and manipulated by the agents. Actions manipulate resources, are linked to a specific agent, and represent the work at its most atomic unit. The work model specifies each action's frequency, priority, duration of resources it needs or manipulates, and which agents are involved [**?**]. Agents serve the dual purpose

of organizing actions and adding a layer of dynamics to the prescribed action sequence by placing limits on both the number of simultaneously performed actions and their priorities [**?**].

A scenario pulls all elements of work models, agents, actions, and resources into a scenario simulation. This can be used to generate an action trace and other higher-level metrics of interest. The simulation engine works on a hybrid timing mechanism that allows WMC to incorporate features of both continuous time and event-based simulation. This enables WMC to simulate dynamic systems (such as aircraft dynamics) and event -based agents (such as pilot models) [**? ?**].

For human agents, WMC can model human taskload [**?**]. Each modeled human agent has two priority queues: one representing actions that are *active* (currently being executed) and one representing actions that are *inactive*. Inactive actions can have two designations. Those that have never been executed are designated as *waiting*, and actions that were previously active but are now inactive are designated as *delayed*. The active queue has a limited capacity which results in actions transitioning between queues. If a human agent is assigned new actions, those actions are put in the inactive queue and given the *waiting* designation. If there is room in the active queue, the highest priority actions (those with the highest explicit priority with the shortest execution time as determined by the action's resources) are moved to the *active* queue. If there are active actions with lower priorities than those in the inactive queue, those lower-priority actions are moved to the *inactive* queue and designated as *delayed*, and the higher priority actions are moved up into the *active* queue. As actions are finished, they are removed from the *active* queue and rescheduled for later (if they occur again) or to never occur again. Within this infrastructure, taskload can be described as the complete set of actions in all of an agent's queues.

III. Formal Modeling Architecture

To formally model these WMC concepts, we have created an abstract architecture (Fig. 2). In this, the formal model is represented by three synchronously composed modules: a Scheduler, Actions, and Agents. The Scheduler keeps track of modeled time, determines when actions are assigned to agents, and coordinates the behavior of the other modules based on the scheduler's status. The Actions module is actually a collection of synchronously composed action submodules that represent actions from the WMC simulation. Similarly, Agents is a collection of synchronously composed agent submodules that represent agents from the WMC simulation.

The Actions and Agents modules communicate with each other via arrays of action and agent data types (*actions* and *agents* from Fig. 2), where each action and agent is associated with an instance of its respective data type. The action data type contains all of the variables shown in Table I, while the agent data type is defined by the variables in Table II.

Our architecture does not explicitly represent the priority queues underlying WMC agents. Rather, the state of these can be inferred by reasoning over the array of action data types. To do this efficiently, we make use of λ calculus operations so that we can reason about sets [**?**] of actions. In this sense, a set is a mapping of action *id*s to Boolean values $actionset : actionID \rightarrow Boolean$. Such sets use λ operations to define this mapping. For example, the empty set $\varnothing = \lambda(i \in actionIDs) : \text{False}$. This can be interpreted as: for all possible values of action id i, i is not in the set (i maps to False).

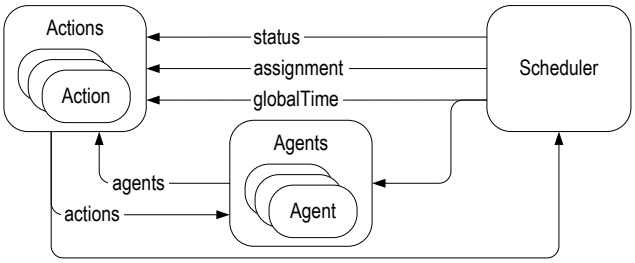

Fig. 2. Formal modeling architecture used to represent WMC concepts.

TABLE I
Variables that Define the Action Data Type

Variable	Description
id	A unique action identification as an integer from 1 to *N*, where there are *N* total actions.
agent	The identification of the agent responsible for the action.
state	The priority queue location and status of the action: whether it is in *active*, *waiting*, or *delayed*, or *notAssigned* (the state of an action that has yet to be assigned or has been finished and rescheduled).
priority	The priority level of the action as a bounded integer.
time	The time left for the action to finish executing (initial times are determined by the time it takes to set an action's resources in the original WMC model).
update	The next time the action will be assigned (this is determined by the observed timings in the simulation trace).

TABLE II
Variables that Define the Agent Data Type

Variable	Description
id	A unique agent identification as an integer from 1 to *M*, where there are *M* total agents.
activeCapacity	The agent's active priority queue capacity.
activeCount	The number of active actions the agent is responsible for (the number of actions in the agent's active priority queue).
minActive	The action data type for an active action the agent is responsible for that has the minimum priority (smallest *priority* and longest *time*) of all such actions.
maxInactive	The action data type for a *waiting* or *delayed* action (an action in the *inactive* queue) the agent is responsible for that has the maximum priority (greatest *priority* and shortest *time*) of all such actions.

Our architecture also abstracts away the WMC concept of resources in service of computational efficiency. However, what resources are being modified at any given time can still be inferred from which actions are executing at that time.

The following describes the details of each of the elements in our architecture.

A. Scheduler

The Scheduler module is responsible for maintaining the clock and communicating the *globalTime* to the other modules.

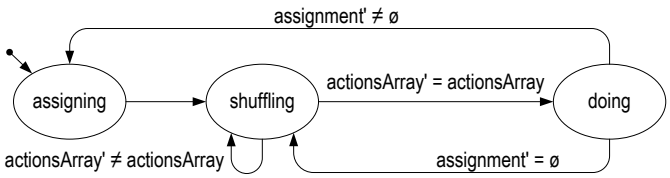

Fig. 3. State transition system representing the scheduler's *status*. Note that an $'$ on a variable indicates that variable's value in the next state. For example $actions' \neq actions$ is checking whether *actions* will change in the next state.

It also indicates when *notAssigned* actions with *update* times at the current *globalTime* are ready to be executed via the *assignment* variable. Finally, the Scheduler uses its *status* to coordinate the behavior of the other modules.

The Scheduler's *status* transitions between its three states using the logic in Fig. 3. Specifically, it starts out *assigning*, where it indicates which actions are ready to be executed. After *assigning*, it automatically transitions to *shuffling*. When the Scheduler is *shuffling*, the action modules are able to reassign the execution state of assigned actions (move them between priority queues). While *shuffling*, the Scheduler monitors the state of *actions* to see if any changes will occur in the next state. If any changes do occur, the Scheduler remains *shuffling*. If there are no changes, the Scheduler status transitions to *doing*. If the Scheduler's *status* is *doing* then, if in the next state there is nothing to assign, the *status* transitions to *shuffling*. Otherwise it transitions to *assigning*.

If the Scheduler is *assigning* it communicates which actions are ready to be performed by computing a λ calculus set. This set is defined as

$$\begin{aligned} assignment = \lambda(i \in actionIDs) : \\ actions[i].update = globalTime \\ \wedge actions[i].state = notAssigned, \end{aligned} \tag{1}$$

This can be interpreted as the set of all action *id*s such that the associated actions are currently *notAssigned* and have an *update* time equal to the current *globalTime*.

The Scheduler uses timed automata [**?** **?**] to represent *globalTime* as a real-valued quantity. If the Scheduler status is *doing* then *globalTime* is increased to the minimum value representing the time it takes to finish any of the *active* actions or the next *update* time of any of the *notAssigned* actions.

B. Agents

The Agents module is a composition of synchronously composed agent submodules, where each agent manages the values of its corresponding agent data type. Conceptually, each agent is responsible for keeping track of the number of its active actions (*activeCount*). It also provides information each action will need for moving between execution states (moving between priority queues) in the form of its *minActive* and *maxInactive* variables.

To compute *activeCount* an agent submodule will use the formula shown in (2) with $actionState = active$. Note that in the formal model, the code for doing this operation is automatically generated with a known bound on the number of possible actions (N). This operation is therefore linear and scales efficiently. It is also important to note that this same equation (2) can be used to compute the number of actions that are in the *waiting* and *delayed* priority queues, even though these are not explicitly represented in the formal model.

To compute *minActive* and *maxInactive*, the agent first uses λ calculus to compute sets containing all of the action *id*s that satisfy the minimum active and maximum inactive criteria (*minActiveSet* and *maxInactiveSet*, respectively). The *minActiveSet* is computed as shown in (3) where, for all action *id*s i in a set of *actionIDs*, i is in the set if the action with $id = i$ is *active*, associated with the given agent, and has a priority less than or equal to all other *active* actions associated with the agent. *maxInactiveSet* is computed as shown in (4) where, for all action *id*s i in a set of *actionIDs*, i is in the set if the action with $id = i$ is *delayed* or *waiting*, associated with the given agent, and has a priority greater than or equal to all other *waiting* or *delayed* actions associated with the agent. With these sets computed, the actions *minActive* and *maxInactive* are selected non-deterministically from the action *id*s in *minActiveSet* and *maxInactiveSet* respectively. This allows for non-determinism in what actions will ultimately be active or inactive at any given time if the actions have the same priority.

C. Actions

Each action within the Actions module is responsible for managing the values in the associated action data type in response to the Scheduler's status and the global time. For any given model state, each action behaves as follows:

- If *status* is *doing* and that action's *state* is *active*, then the action's *time* is decremented based on the amount elapsed since the clock was last updated. If doing this means that the action has finished (that *time* becomes 0), the action's *state* is set to *notAssigned* and its *update* time is set to the action's next update time from the original simulation trace. To add non-determinism to the timing of actions, variance can be included in the update time.
- If *status* is *assigning* and the action is in the set of assigned actions, then the action's *state* is set to *waiting* and the action's *time* is updated. Non-deterministic amounts of time variance can also be added in this assignment.
- If *status* is *shuffling*, then:
  - If the action's state is *waiting* or *delayed* and it is equal to its agent's *maxInactive* action and either the action has a higher priority than its agent's *minActive* action or its agent's active capacity has not been reached, then the action's state is set to *active*.
  - If the action's state is *active* and it is equal to its agent's *minActive* action and the agent's active capacity has been exceeded, then the action's state is set to *delayed*.

D. Analysis Capabilities

Our architecture gives us the ability to model sections of simulation traces with included variance in the timing of actions. This is useful because it gives us the ability to reason about taskload in specification properties that allow us to assert the absences of potentially problematic conditions for human

$$\begin{aligned} cardinality(actionState) = &\begin{cases} 1, & \text{if } actions[1].agent = agentID \wedge actions[1].state = actionState \\ 0, & \text{otherwise} \end{cases} \\ &+ \ldots + \begin{cases} 1, & \text{if } actions[N].agent = agentID \wedge actions[N].state = actionState \\ 0, & \text{otherwise} \end{cases} \end{aligned} \tag{2}$$

$$\begin{aligned} minActiveSet = &\lambda(i \in actionIDs) : actions[i].agent = agentID \wedge actions[i].state = active \\ &\wedge \forall (j \in actionIDs) : \\ &\left(\begin{pmatrix} actions[j].agent = agentID \\ \wedge actions[j].state = active \end{pmatrix} \Rightarrow \begin{pmatrix} actions[i].priority < actions[j].priority \\ \vee \begin{pmatrix} actions[i].priority = actions[j].priority \\ \wedge actions[i].time > actions[j].time \end{pmatrix} \end{pmatrix} \right) \end{aligned} \tag{3}$$

$$\begin{aligned} maxInactiveSet = &\lambda(i \in actionIDs) : actions[i].agent = agentID \wedge (actions[i].state = waiting \vee actions[i].state = delayed) \\ &\wedge \forall (j \in actionIDs) : \\ &\left(\begin{pmatrix} actions[j].agent = agentID \\ \wedge \begin{pmatrix} actions[j].state = waiting \\ \vee actions[j].state = delayed \end{pmatrix} \end{pmatrix} \Rightarrow \begin{pmatrix} actions[i].priority < actions[j].priority \\ \vee \begin{pmatrix} actions[i].priority = actions[j].priority \\ \wedge actions[i].time < actions[j].time \end{pmatrix} \end{pmatrix} \right) \end{aligned} \tag{4}$$

agents. We can then use these to generate counterexamples, allowing us to use our method (Fig. 1) to create WMC scenarios to examine the conditions found in the counterexample in the simulation. For our current purposes, we are interested in specifications that concern excessive workload conditions and conditions where people may drop or forget actions.

For excessive workload, we are concerned with finding conditions where the human operator's active priority queue is at capacity [**?**]. We can use linear temporal logic to assert that the active queue for a given human agent with $id = i$ will never reach capacity with

$$\mathbf{G}\neg \left(\begin{pmatrix} (status = doing) \\ \Rightarrow \begin{pmatrix} agent[i].activeCapacity \\ \neq agent[i].activeCount \end{pmatrix} \end{pmatrix} \right), \tag{5}$$

where this can be interpreted as: for all paths through the model ($\mathbf{G}$) we never want it to be true that if the scheduler *status* is *doing*, then agent i's *activeCount* reaches or exceeds its *activeCapacity*. Note that we are only concerned with the capacity of an agent's queues when the scheduler's *status* is *doing* because, by design, queue capacities may be exceeded during nominal *assigning* and *shuffling* operations.

By adding a synchronous observer (see [**?**]), we can also have a model variable (*count*) that can count the number of clock update periods over which a full active queue is maintained. This can allow us to specify that an active queue should never remain full for over K updates of the clock as

$$\mathbf{G}\neg (count \geq K). \tag{6}$$

Thus, we can generate traces illustrating how a human agent remains at maximum active capacity over K periods.

There are several different reasons why a human operator may fail to perform an action. First, if the human's working memory is exceeded (indicated by excessive actions that are *waiting* or *delayed*) he or she might forget an action. We can assert the absence of this condition as follows:

$$\mathbf{G}\neg \left(\begin{pmatrix} (status = doing) \\ \Rightarrow \left(\begin{pmatrix} cardinality(waiting) \\ + cardinality(delayed) \end{pmatrix} \geq max \right) \end{pmatrix} \right), \tag{7}$$

where *max* is the maximum capacity of the *inactive* queue.

A human may also forget an action if it remains in working memory (*waiting* or *delayed*) for too long [**?**]. We can specify that this should never occur as

$$\mathbf{G}\neg \left(\begin{pmatrix} (actions[j].state \neq notAssigned) \\ \Rightarrow \left(\begin{pmatrix} globalTime \\ - actions[j].update \end{pmatrix} \geq timeMax \right) \end{pmatrix} \right), \tag{8}$$

where *timeMax* is an analyst-specified waiting time that an action should not exceed.

IV. Discussion

The presented architecture allows us to formally represent all of the relevant WMC concepts while satisfying our objectives. (a) It uses timed automata to represent real-valued time and allows for sensitivity analyses of WMC concepts based on variance in the timing of actions; (b) It allows taskload to be modeled by having the model reason over an array of actions, each with its own state and associated with a different agent; and (c) By using λ operations over the set of actions and by abstracting away WMC details unimportant to the formal analyses, the architecture is computationally efficient.

Additionally, a number of specification properties can be used to reason about taskload for use in generating counterexamples for later use in scenario creation. We currently have a working version of this architecture operating with the WMC simulation and the infinite bounded model checker in SAL.

This work was only focused on the architecture required to realize our method (Fig. 1). Future work will complete the method and use it to evaluate realistic air traffic scenarios.

A. Translation Processes

We have already developed a prototype translation process that converts WMC trace, scenario, and work model information into a formal model that uses the architecture presented here. This method is currently being used to evaluate existing WMC models and scenarios. Future work will focus on refining this translation process to reduce the amount of human analyst intervention required. Additionally, in current efforts, all reverse translation (Translation 2 from Fig. 1) must be done manually. Future efforts will focus on automating this translation process. As this project progresses, these translations will benefit from the use of standardized WMC XML models.

B. XML Models

The current implementation of the method evaluates SAL models generated from WMC action traces. In future work, we hope to rely less on action traces and offer a flexible mechanism for developing scenarios and simulations with an XML description of the work model and scenario. The XML Specification would create a universal starting point for modeling new systems of interest and allow rapid changes to existing work models. Having such an abstracted specification would support collaborators or general users who are unfamiliar with WMC or SAL but more familiar with XML markup. Additionally, an XML Specification would allow for XML files of work models and scenarios to be translated back and forth between WMC simulations and SAL experiments more easily. The XML Parser currently translates XML into WMC, and other directions are being developed now.

C. Realistic Application

We will use the completed method (Fig. 1) to evaluate realistic air traffic scenarios. In particular, we will examine the different scenarios from [**?**] which represent three simulated aircraft arriving into Amsterdam Airport Schiphol RWY18R under different distributions of authority, autonomy, and responsibility between air and ground-based operators. In particular, the scenarios explored in [**?**] did not take the limitations of human operator taskload into account. Thus, our future efforts will use the formal architecture here and the associated formal verification from the method to generate scenarios that will allow the simulation to explore how performance changes based on different operator memory limitations (priority queue capacity restrictions) and different amounts of variance between the timing of actions.

ACKNOWLEDGEMENT

This work was supported by the grant “Scenario-Based Verification and Validation of Autonomy and Authority” from the NASA Ames Research Center under award number NNX13AB71A.